Microsoft

# Whats New in PostgreSQL 19

Daniel Gustafsson

2026-06-16 - Lund, Sweden




Previously

vmware™ Pivotal

https://archive.org/details/office-43




https://archive.org/details/office-43

*User since 2005*

*Contributing since ~ 2013*

*Major Contributor and Committer (`postgres/pgweb`)*

*PGEvents Canada Board of directors*

# POSTGRES DEVELOPMENT

# History

*Research project at Berkeley in 1986 to address the main shortcoming of INGRES - extensibility*

*Open sourced 30 years ago when research ended. Carefully set up to be autonomous and safe from any takeover or acquisition*

# Development Practice

*Written in C, tests in Perl and Python*

*Support Linux, AIX, *BSD, macOS and Windows*

*Linear version control history, no merge commits*

*No governance, no shared roadmap, no hierarchy*

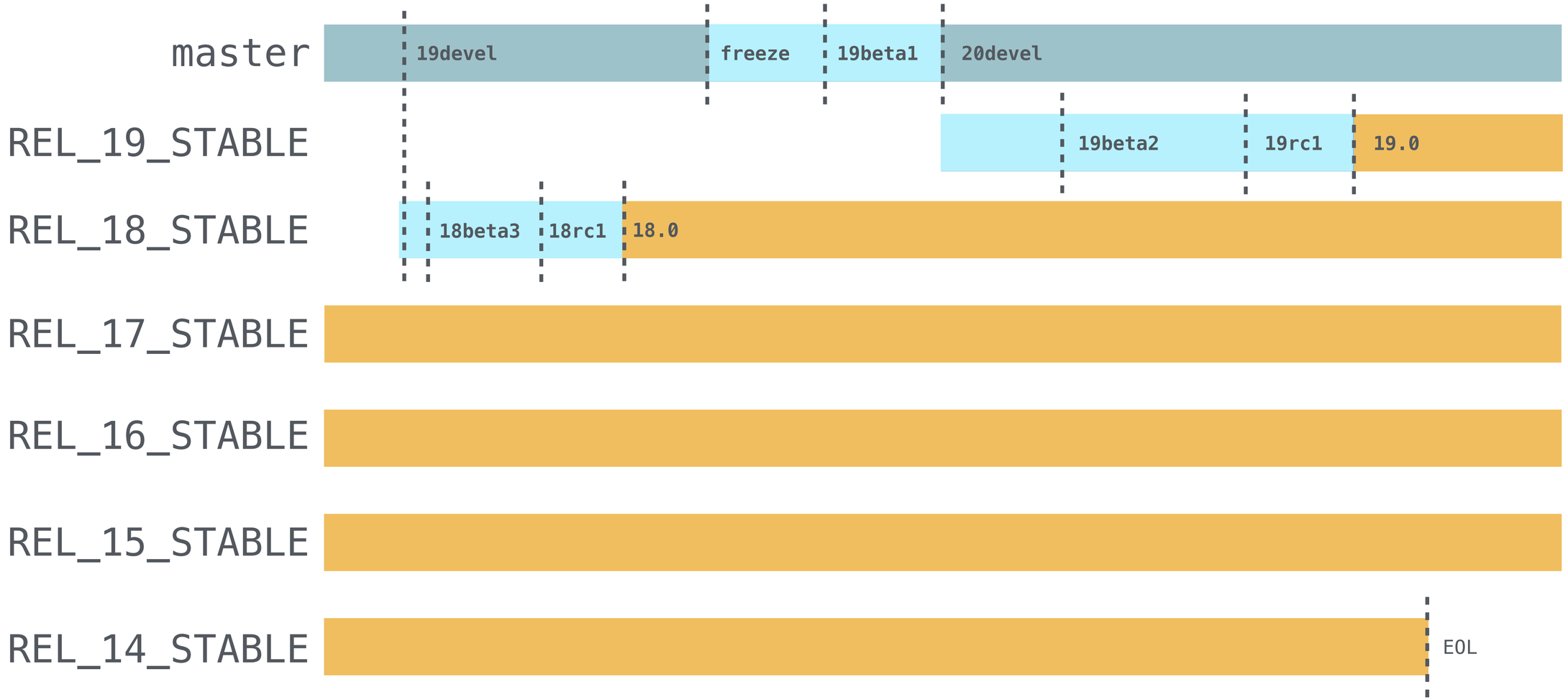

databricks
The data + AI
mpany
PostgreSQL
PostgreSQL
PGConf
.dev

```
commit 2652835d3efa003439ecc23d5fc3cf089c5952a6
Author: Joe Conway <mail@joeconway.com>
Date:   Sun Jun 29 22:28:10 2025 -0400

    Stamp HEAD as 19devel.

    Let the hacking begin ...
```

# PostgreSQL Development Cycle



```
$ git diff --stat 2652835.. |tail -1|sed 's/,/\n/g'
4477 files changed
528677 insertions(+)
324834 deletions(-)

$ git log 2652835.. |grep "^commit"|wc -l
    2893
```

*.. as of 2026-06-15*

× Performance

× Developer Experience

× Operations / Management

× Security

# OPERATIONS MANAGEMENT

# Online Data Checksums

*Until v18, set at* `initdb` *time or offline with* `bin/pg_checksums`*, online capabilities in v19*

*On by default since v18 (YMMV in distributions)*

*Verified when reading, calculated when writing*

# Online Data Checksums

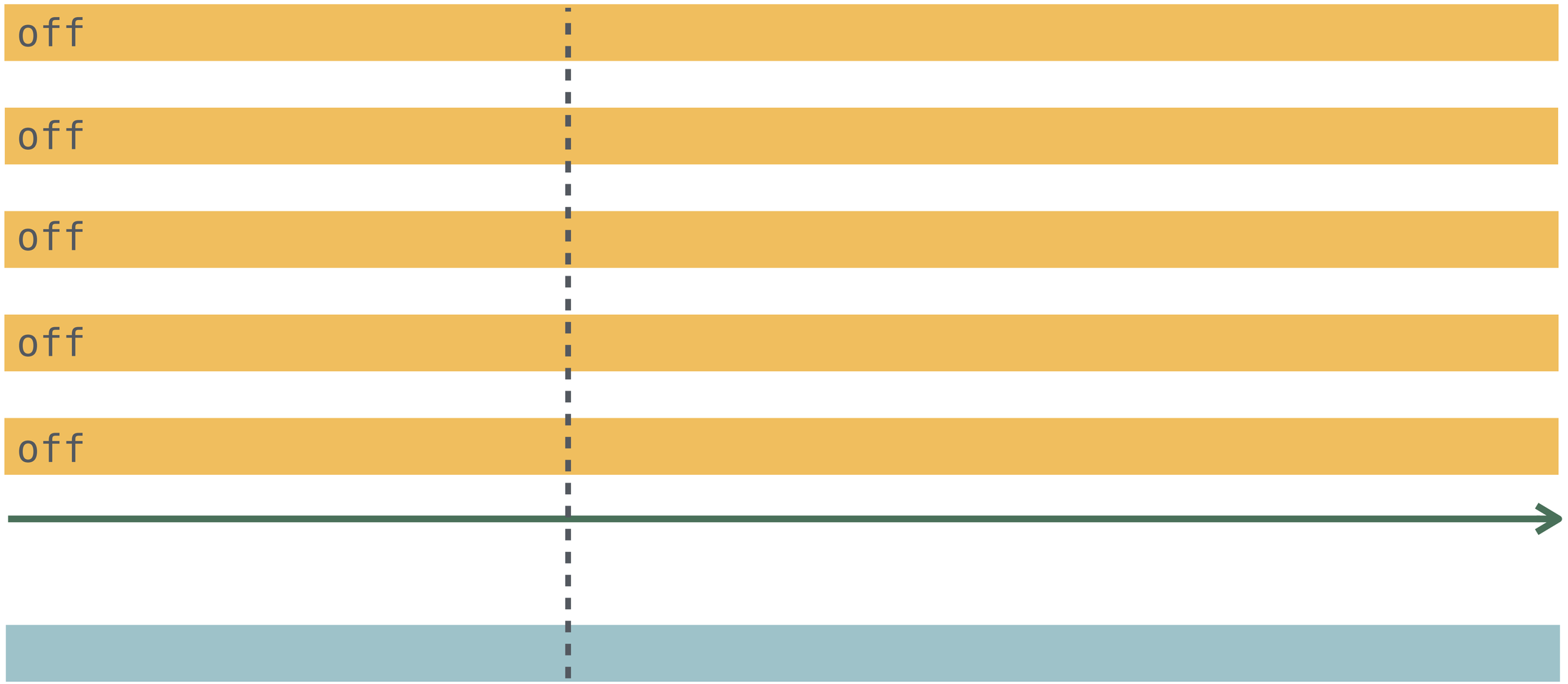
*No restrictions on running cluster, impact is mainly I/O. Cost throttling to mitigate*

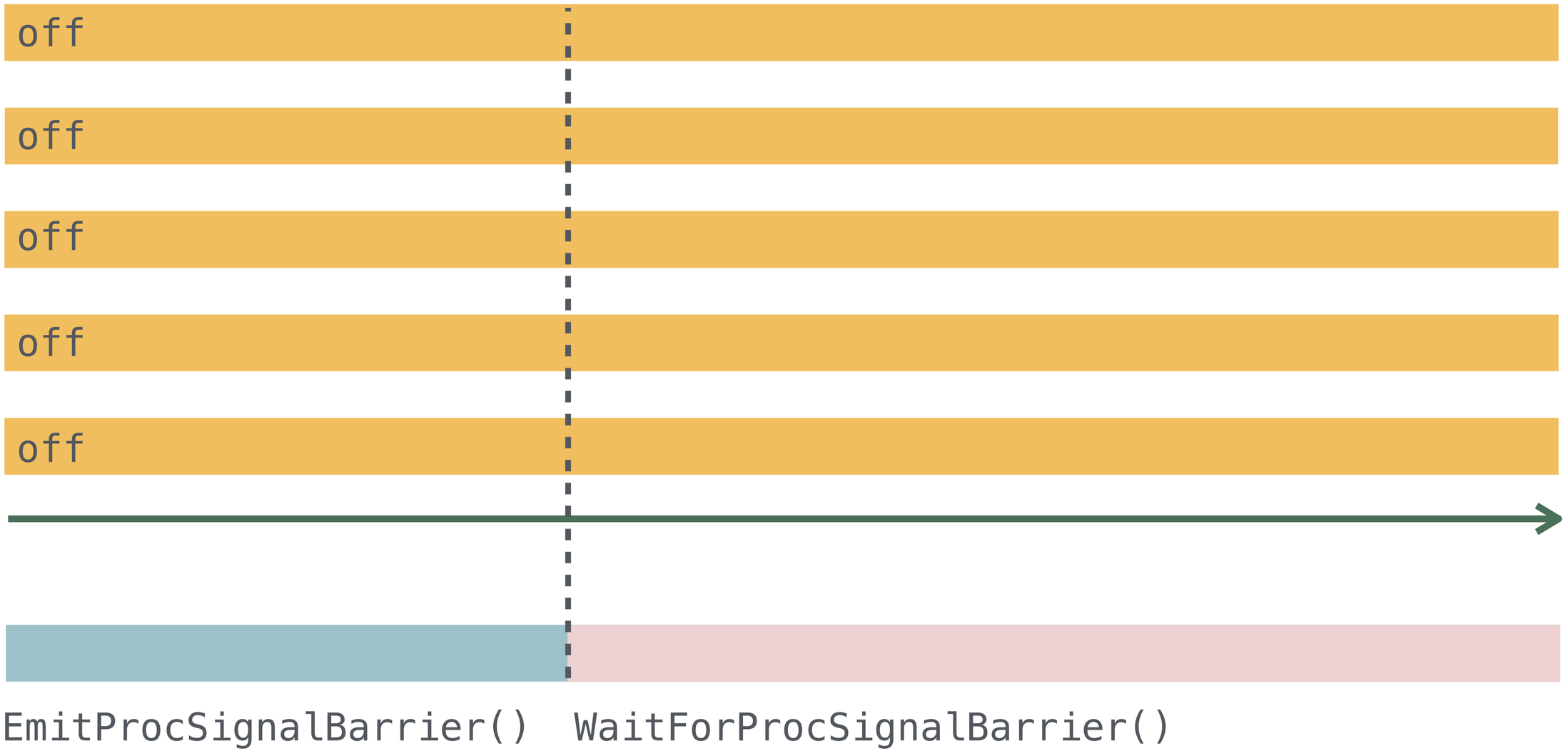
*New states for* `data_checksums` *GUC*

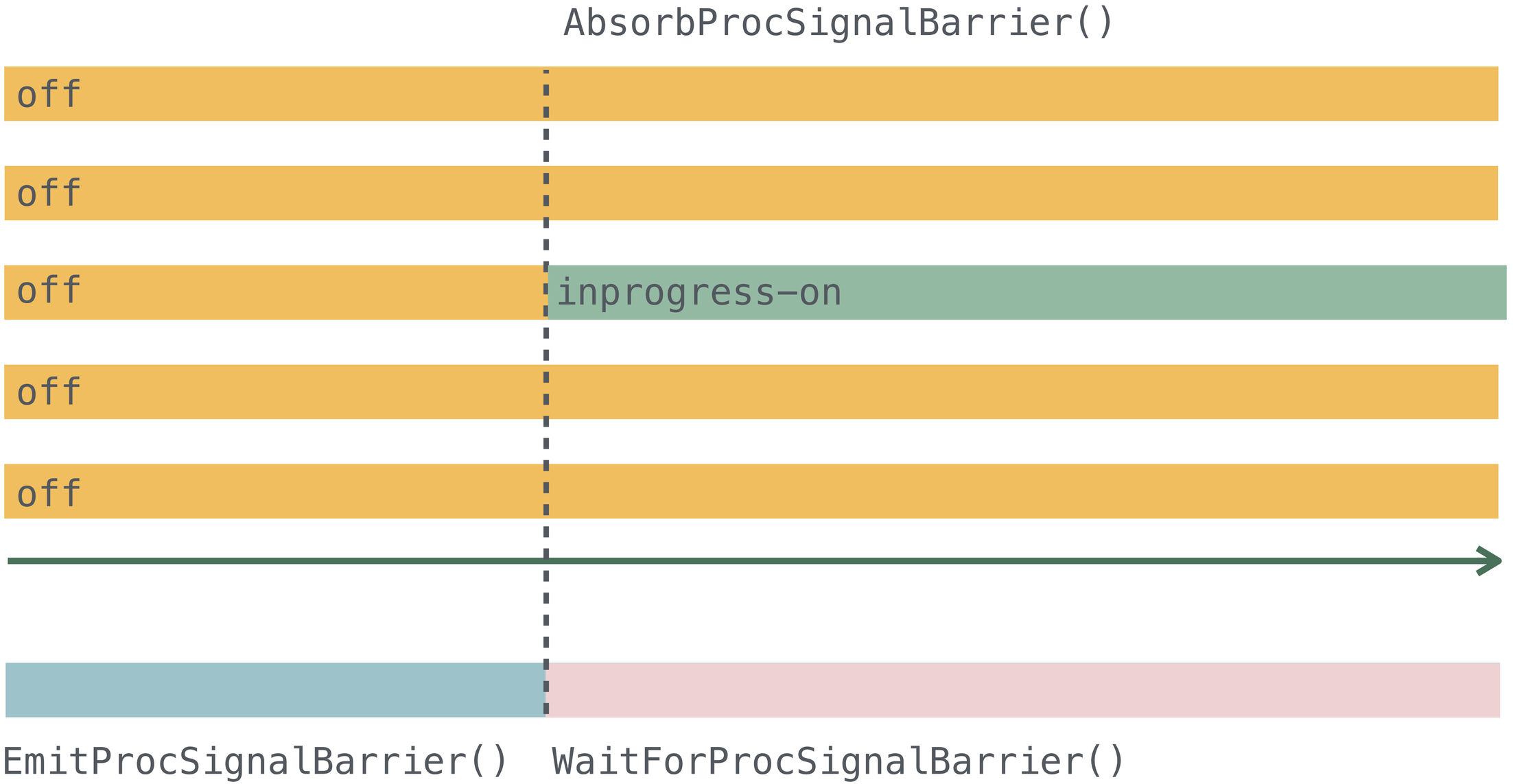


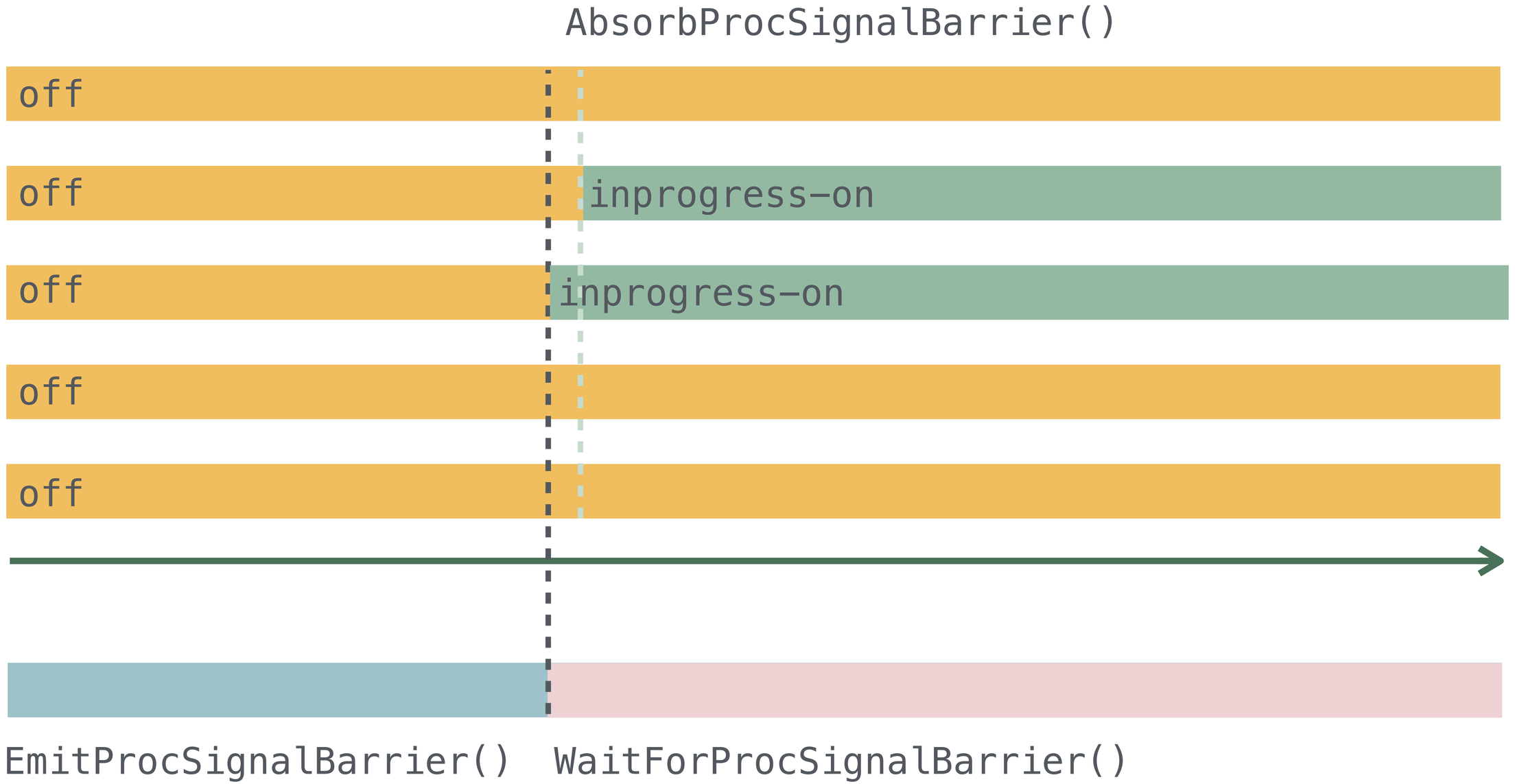

*WAL logged to handle replay, PITR, replication*

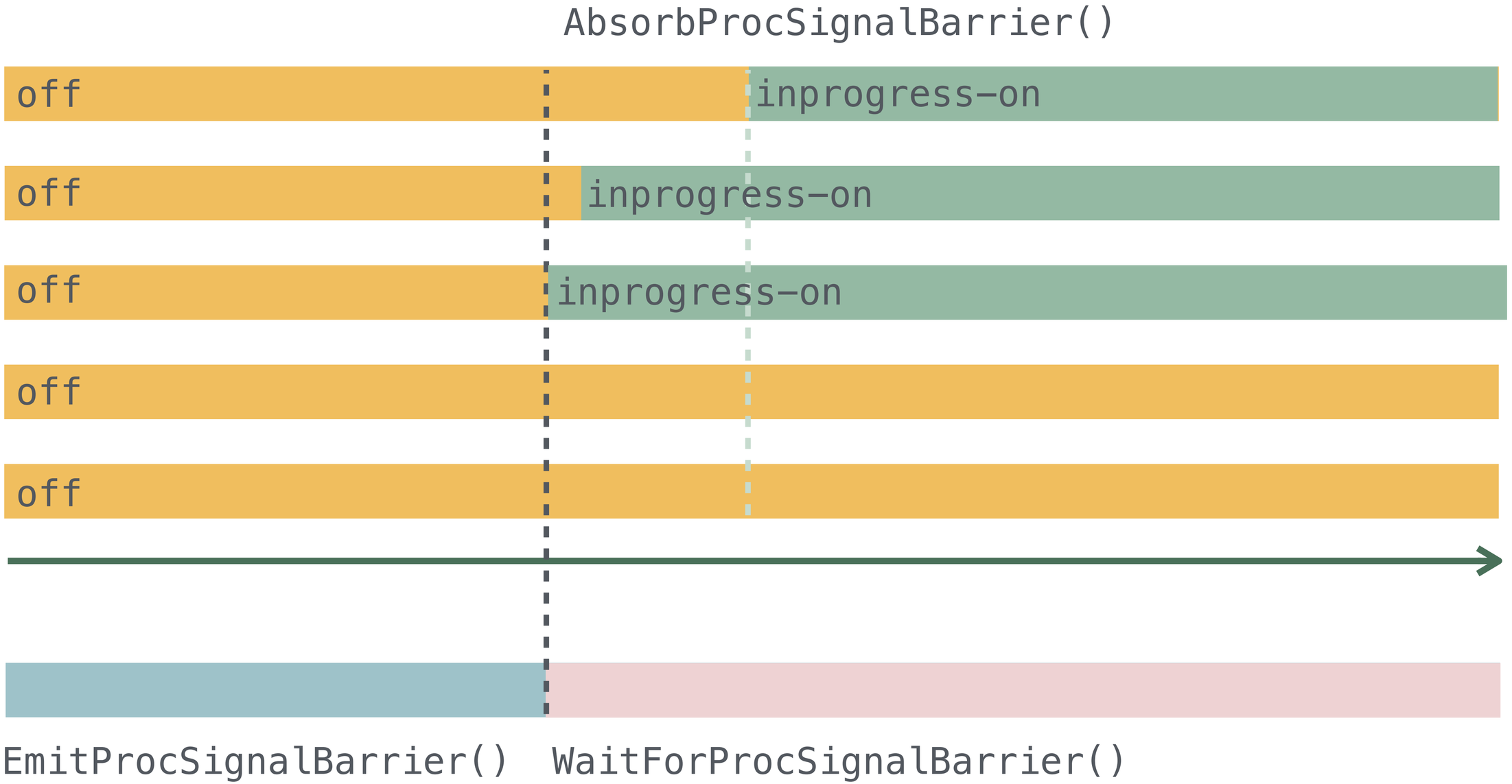

off
off
off
off
off

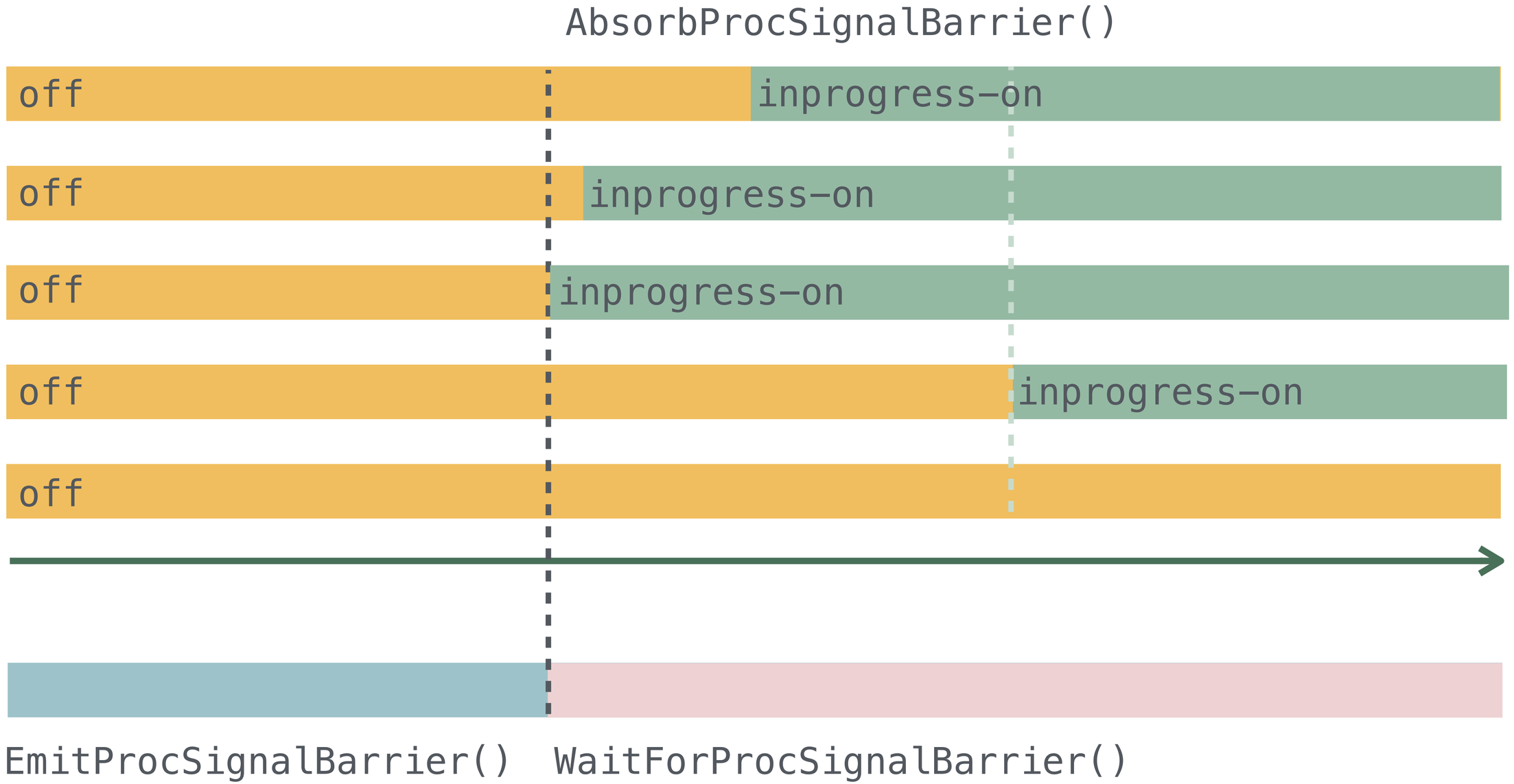

off
off
off
off
off
EmitProcSignalBarrier()

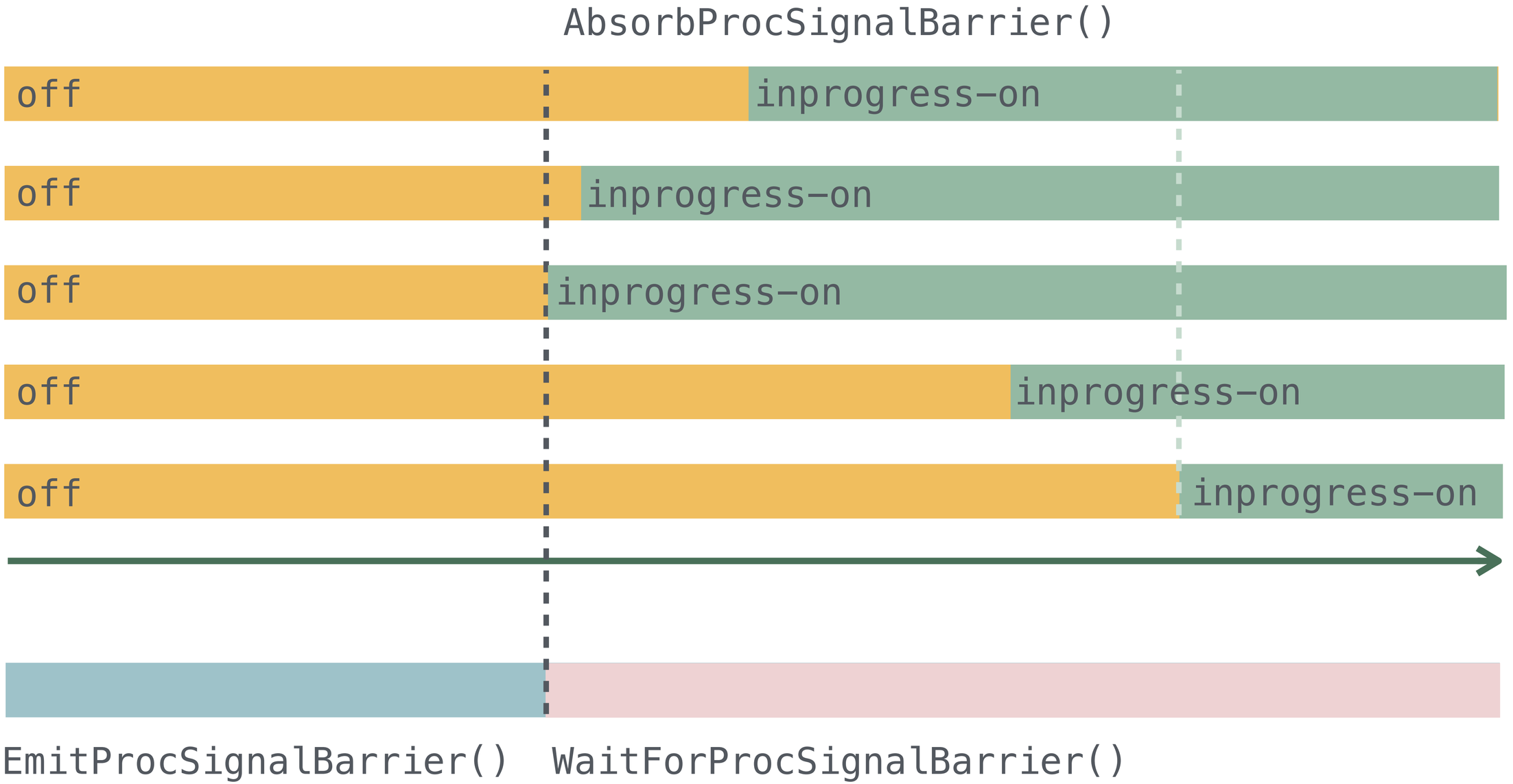

off
off
off
off
off
EmitProcSignalBarrier()
WaitForProcSignalBarrier()

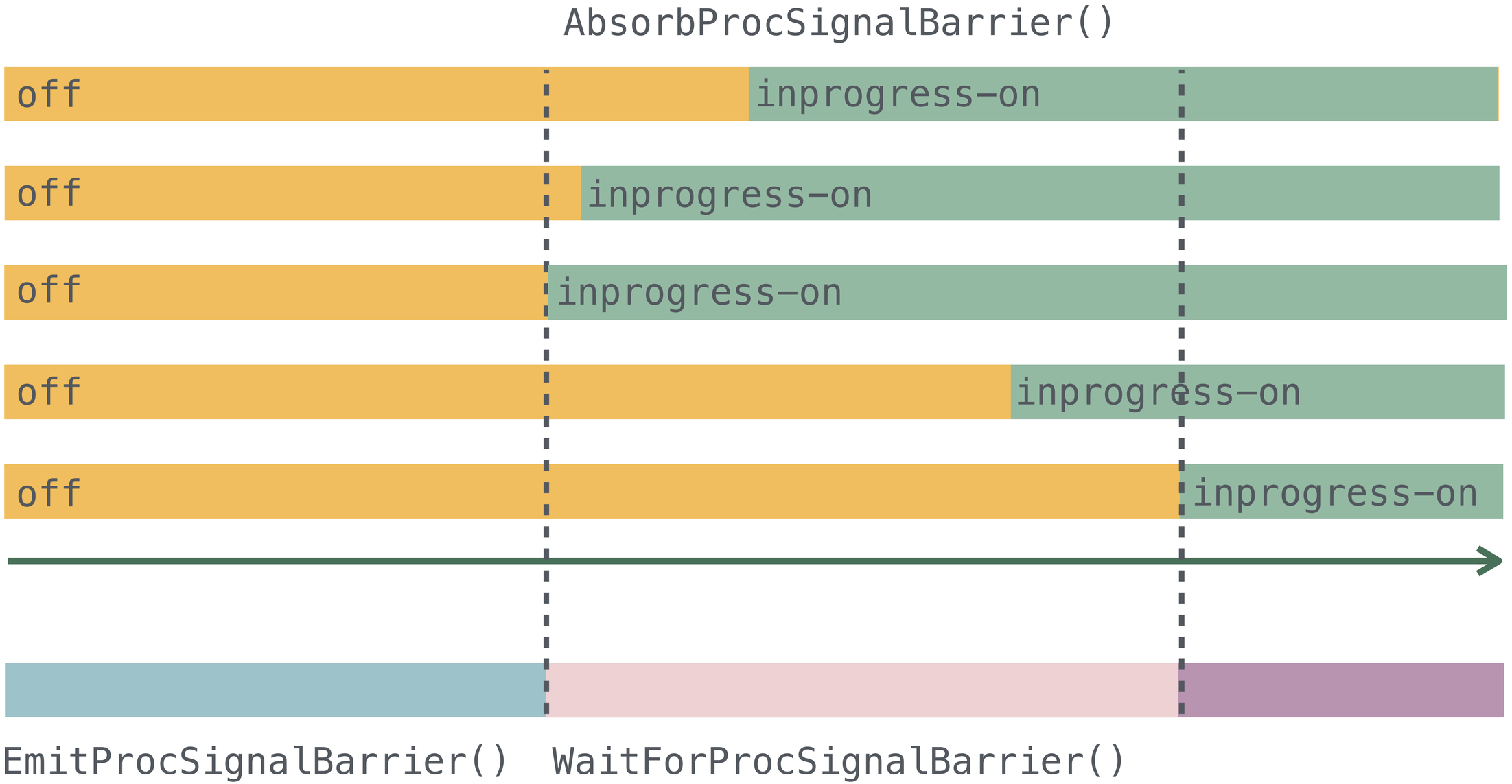

AbsorbProcSignalBarrier()
off
off
off
inprogress-on
off
off
EmitProcSignalBarrier()
WaitForProcSignalBarrier()


AbsorbProcSignalBarrier()
off
off
inprogress-on
off
inprogress-on
off
off
EmitProcSignalBarrier()
WaitForProcSignalBarrier()


AbsorbProcSignalBarrier()
off
inprogress-on
off
inprogress-on
off
inprogress-on
off
off
EmitProcSignalBarrier()
WaitForProcSignalBarrier()


AbsorbProcSignalBarrier()
off
inprogress-on
off
inprogress-on
off
inprogress-on
off
inprogress-on
off
EmitProcSignalBarrier()
WaitForProcSignalBarrier()


AbsorbProcSignalBarrier()
off
inprogress-on
off
inprogress-on
off
inprogress-on
off
inprogress-on
off
inprogress-on
EmitProcSignalBarrier()
WaitForProcSignalBarrier()


AbsorbProcSignalBarrier()
off
inprogress-on
off
inprogress-on
off
inprogress-on
off
inprogress-on
off
inprogress-on
EmitProcSignalBarrier()
WaitForProcSignalBarrier()

```
postgres=# begin;
BEGIN
postgres=*# create temporary
table t(a integer);

CREATE TABLE
```

```
postgres=# show data_checksums;
 data_checksums
----------------
 off
(1 row)

postgres=# select pg_enable_data_checksums();
 pg_enable_data_checksums
--------------------------


(1 row)

postgres=# show data_checksums;
 data_checksums
----------------
 inprogress-on
(1 row)
```

```
postgres=# select wait_event, query
from pg_stat_activity
where backend_type = 'datachecksums launcher';

-[ RECORD 1 ]----------------------------------------
wait_event | ChecksumEnableStartcondition
query      | Waiting for transactions older than 695 to end
```

```
postgres=# commit;

COMMIT
```

```
postgres=# show data_checksums;
 data_checksums
----------------
 inprogress-on
(1 row)
```

```
postgres=# select wait_event, query
from pg_stat_activity
where backend_type = 'datachecksums launcher';

-[ RECORD 1 ]----------------------------------------
wait_event | ChecksumEnableTemptableWait
query      | Waiting for 1 temp tables to be removed
```

```
postgres=# drop table t;

DROP TABLE
```

```
postgres=# show data_checksums;
 data_checksums
----------------
 on
(1 row)
```

# AIO

*The I/O worker pool previously had to be pre-defined, which is hard to guesstimate. Now the pool is automatically sized dynamically between* `io_min_workers` *and* `io_max_workers`

*Queue grows when latency would be incurred by queuing requests*

# Operational Grab Bag

Log levels can be specified per process type with `log_min_messages` accepting a type:level tuple

```
log_min_messages=error, walsender:debug1
```

New system views `pg_stat_lock` for locktype statistics and `pg_stat_recovery` for recovery information. New columns on many other views

# *PERFORMANCE*

# AIO Performance Improvements

*Buffer manager improvements paving the way for using AIO for writes*

*Atomic un-pinning and un-locking buffers show up to ~20% improvement in pgbench*

*Reduced page copying*

# Vacuum

*Parallel autovacuum operation with multiple workers.* `autovacuum_max_parallel_workers` *limits how many workers can be spawned*

*Visibility Map updates during* `SELECT` *queries allows index-only queries without waiting for* `VACUUM`*, and less work to perform during vacuuming*

# REPACK

*Reclaim storage from dead tuples by rewriting the database files, unlike* `VACUUM`

*Does not hold an ACCESS EXCLUSIVE lock on the relation, unlike* `VACUUM FULL`

```
postgres=# create table t (a integer primary key, b text);
CREATE TABLE
postgres=# insert into t
select generate_series(1,1000000) a,
       generate_series(1,1000000)::TEXT b;
INSERT 0 1000000
postgres=# select pg_size_pretty(pg_total_relation_size(oid))
           from pg_class where relname = 't';
 pg_size_pretty
----------------
 64 MB
(1 row)
```

```
postgres=# update t set b = repeat(b, 10);
UPDATE 1000000
postgres=# select pg_size_pretty(pg_total_relation_size(oid))
           from pg_class where relname = 't';
 pg_size_pretty
----------------
 180 MB
(1 row)
postgres=# delete from t where a % 2 = 0;
DELETE 500000
postgres=# select pg_size_pretty(pg_total_relation_size(oid))
           from pg_class where relname = 't';
 pg_size_pretty
----------------
 180 MB
(1 row)
```

```
postgres=# vacuum full t;
VACUUM
postgres=# select pg_size_pretty(pg_total_relation_size(oid))
           from pg_class where relname = 't';
 pg_size_pretty
----------------
 58 MB
(1 row)
```

*ACCESS EXCLUSIVE lock held*
*for the duration of the operation*

```
postgres=# repack (concurrently) t;
REPACK
postgres=# select pg_size_pretty(pg_total_relation_size(oid))
           from pg_class where relname = 't';
 pg_size_pretty
----------------
 58 MB
(1 row)
```

*Concurrent activity on the relation allowed, except for a brief window*

# Performance Grab Bag

*Foreign key constraint checks are now up to 2.9x faster*

*Improved tuple deformation, especially for wide tables with fixed-width columns*

*Adding columns with non-volatile domains using constraints no longer require a table rewrite*

# Performance Grab Bag

*TOAST compression is now using LZ4 by default, as long as the server is compiled with liblz4*

`NOTIFY` *now only wakes up backends listening to the channel where the message is sent which improves throughput greatly and reduces latency for large numbers listeners/channels*

# Performance Grab Bag

*Crypto Extensions on ARM and AVX2 CPU instructions on Intel/AMD used for calculating checksums*

*Eager aggregation push partial aggregation before joins, and finalize after join, which can reduce number of rows needing to be joined*

# SECURITY

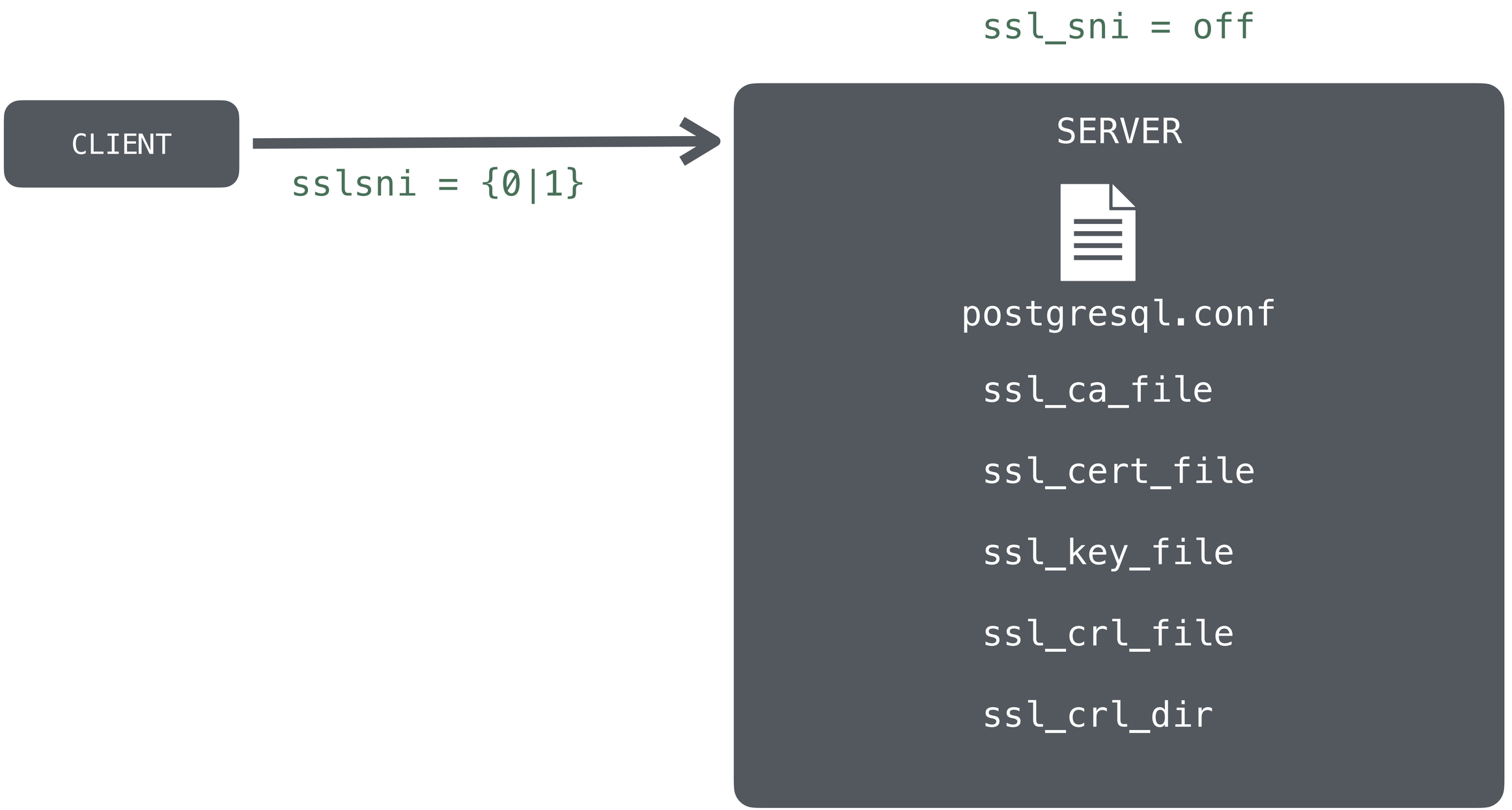
# Host Based Multitenancy

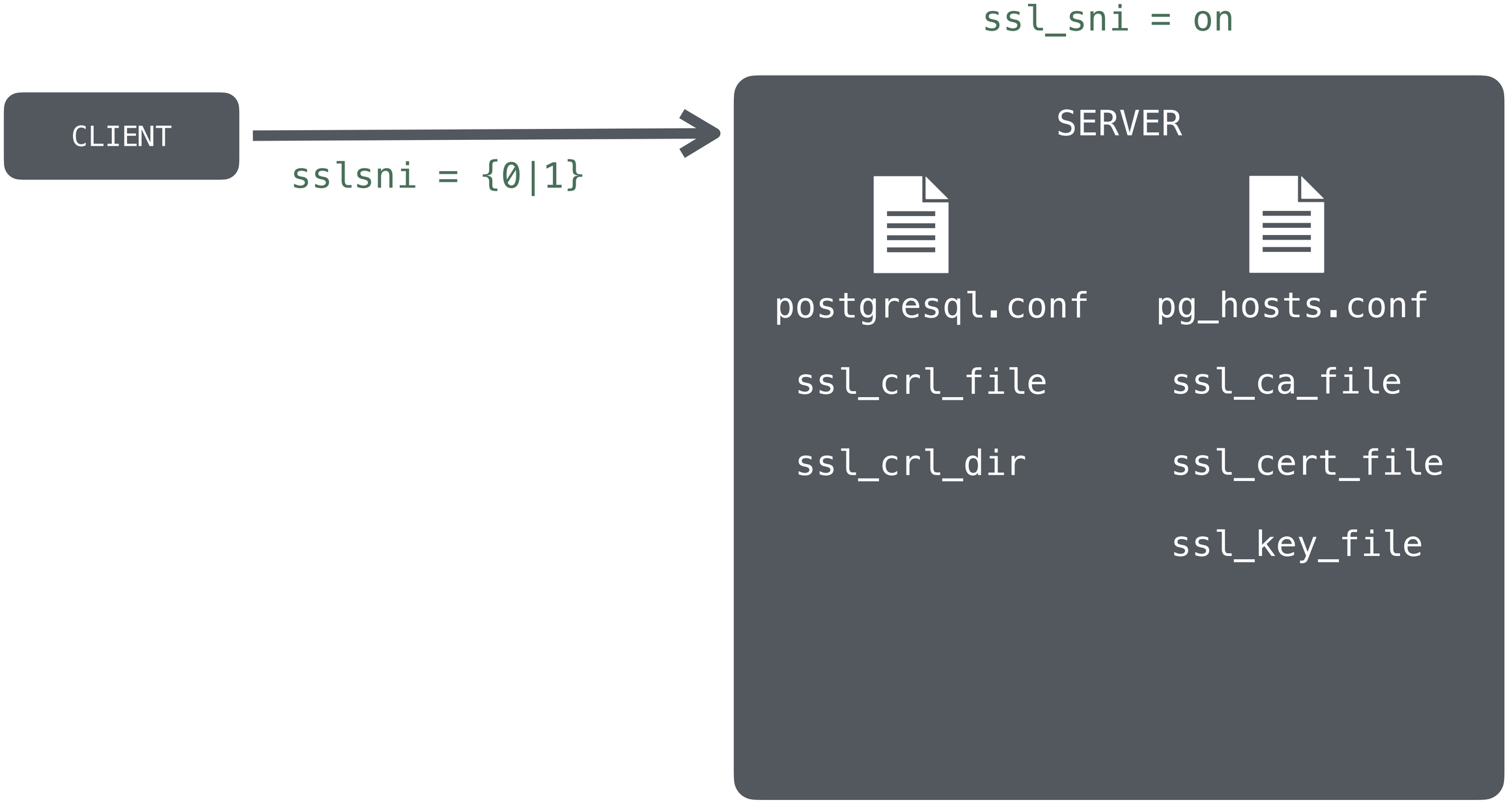
*Serverside support for Server Name Indication TLS extension*

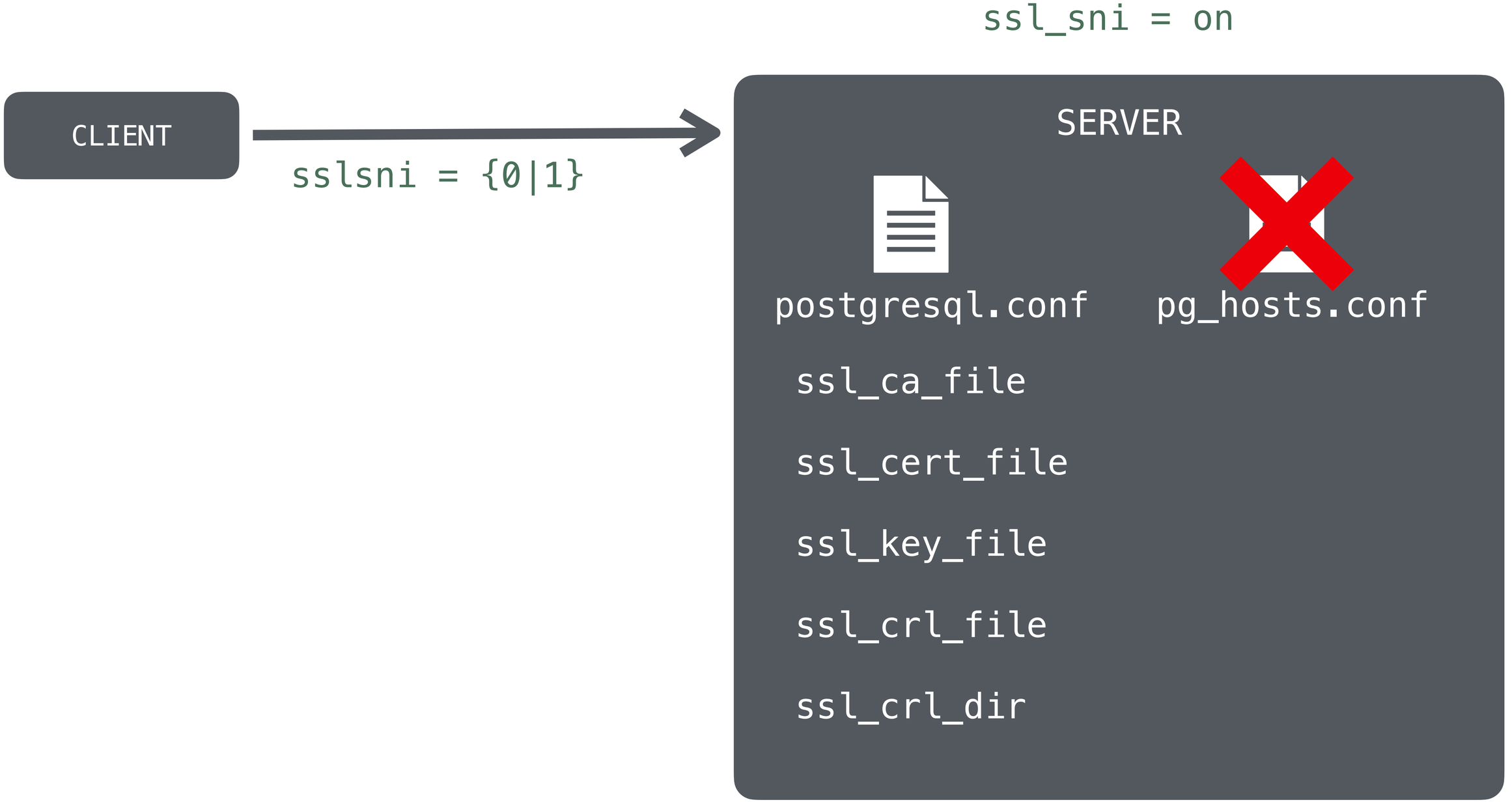
*Clientside support in all supported versions*

*New configure file* `$DATADIR/pg_hosts.conf`

*Fully backwards compatible, no configuration change required to keep current model*


ssl_sni = off
CLIENT
sslsni = {0|1}
SERVER
postgresql.conf
ssl_ca_file
ssl_cert_file
ssl_key_file
ssl_crl_file
ssl_crl_dir


ssl_sni = on
CLIENT
sslsni = {0|1}
SERVER
postgresql.conf
ssl_crl_file
ssl_crl_dir
pg_hosts.conf
ssl_ca_file
ssl_cert_file
ssl_key_file


ssl_sni = on
CLIENT
sslsni = {0|1}
SERVER
postgresql.conf
pg_hosts.conf
ssl_ca_file
ssl_cert_file
ssl_key_file
ssl_crl_file
ssl_crl_dir

```
# PostgreSQL SNI Hostname mappings
# ================================

# HOSTNAME    SSL CERTIFICATE  SSL KEY       SSL CA    PASSPHRASE COMMAND    PASSPHRASE COMMAND RELOAD
*             default.cert     default.key   server.ca
foo.com       foo.cert         foo.key       wan.ca
bar.com       bar.cert         bar.key       wan.ca    "echo SeCrEt!"        Yes
alice         alice.cert       alice.key     private.ca
bob           bob.cert         bok.key       private.ca
/no_sni/      nosni.cert       nosni.key     server.ca
```

# OAuth

*OAuth validators can now register custom* `pg_hba` *authentication options*

*OAuth validators can now report custom failures*

*New connection parameter* `oauth_ca_file` *to set CA's outside of system trust store, for container usecases (mainly)*

# Security Grab Bag

MD5 authentication deprecation continues, a WARNING will be issued when authenticating with MD5. `md5_password_warning` can be used to turn it off

`password_expiration_warning_threshold` can be enabled to issue a WARNING for VALID UNTIL roles which are about to expire

# Security Grab Bag

*CVE-2026-2006: Reject incomplete or truncated multibyte characters instead of silently ignoring*

*Names of relations, roles and namespaces are no longer allow to contain linebreaks*



# DEVELOPER EXPERIENCE

# `COPY` *Improvements*

*No need to wrap selects with JSON constructors,* `COPY .. TO` *can output natively in NDJSON format. Optionally wrapped in an array*

`COPY table TO` *works on partitioned tables*

`COPY FROM` *optimised to use SIMD instructions for text/CSV parsing. Up to 60% improvement*

```
postgres=# create view v as select oid, relname, relhasindex from pg_class;

CREATE VIEW

postgres=# copy (select * from v limit 2) to stdout with (format json);

{"oid":"2619","relname":"pg_statistic","relhasindex":true}
{"oid":"1247","relname":"pg_type","relhasindex":true}


postgres=# copy (select * from v limit 2) to stdout with (format json,
force_array);

[
 {"oid":"2619","relname":"pg_statistic","relhasindex":true}
,{"oid":"1247","relname":"pg_type","relhasindex":true}
]
```

# `ON CONFLICT .. DO SELECT`

*Inserting, or updating, a row and getting the result back (with generated columns) is a common pattern*

*Until now, a 100% match caused a data update with associated bloat and WAL activity*

*PostgreSQL v19 adds* `DO SELECT` *to solve this*

```
postgres=# create table users (id integer primary key,
                               uname text unique);
CREATE TABLE
postgres=# insert into users values (1, 'Alice'),
                                    (2, 'Bob');
INSERT 0 2

postgres=# insert into users (id, uname) values (3, 'Alice')
           on conflict (uname) do nothing returning *;
 id | uname
----+-------
(0 rows)

INSERT 0 0
```

```
postgres=# insert into users (id, uname) values (3, 'Alice')
           on conflict (uname) do nothing returning *;
 id | uname
----+-------
(0 rows)
INSERT 0 0

postgres=# insert into users (id, uname) values (3, 'Alice')
           on conflict (uname) do update set uname = excluded.uname
           returning *;
 id | uname
----+-------
  1 | Alice
(1 row)
INSERT 0 1
```

```
postgres=# insert into users (id, uname) values (3, 'Alice')
           on conflict (uname) do nothing returning *;
 id | uname
----+-------
(0 rows)
INSERT 0 0

postgres=# insert into users (id, uname) values (3, 'Alice')
           on conflict (uname) do select returning *;
 id | uname
----+-------
  1 | Alice
(1 row)
INSERT 0 1
```

# UPDATE .. FOR PORTION OF

*PostgreSQL v18 allowed for defining temporal constraints using* `WITHOUT OVERLAP`

*Modifying ranges needed application side logic*

*PostgreSQL v19 adds support for temporal updates with* `UPDATE .. FOR PORTION OF`

```
postgres=# create table oncall (dept int not null,
                                shift tstzrange not null,
                                employee int not null,
                                primary key (dept, shift without overlaps));
CREATE TABLE

postgres=# insert into oncall values
(1, tstzrange('2026-06-01', '2026-06-07'), 10),
(1, tstzrange('2026-06-08', '2026-06-14'), 20),
(1, tstzrange('2026-06-15', '2026-06-21'), 30);
INSERT 0 3
postgres=# insert into oncall values
(2, tstzrange('2026-06-01', '2026-06-07'), 40),
(2, tstzrange('2026-06-08', '2026-06-14'), 50),
(2, tstzrange('2026-06-15', '2026-06-21'), 60);
INSERT 0 3
```

```
postgres=# select * from oncall;
 dept |                              shift                              | employee
------+-----------------------------------------------------------------+----------
    1 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02") |       10
    1 | ["2026-06-08 00:00:00+02","2026-06-14 00:00:00+02") |       20
    1 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02") |       30
    2 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02") |       40
    2 | ["2026-06-08 00:00:00+02","2026-06-14 00:00:00+02") |       50
    2 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02") |       60
(6 rows)
```

```
postgres=# select * from oncall;
 dept |                                shift                                 | employee
------+----------------------------------------------------------------------+----------
    1 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02")                  |       10
    1 | ["2026-06-08 00:00:00+02","2026-06-14 00:00:00+02")                  |       20
    1 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02")                  |       30
    2 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02")                  |       40
    2 | ["2026-06-08 00:00:00+02","2026-06-14 00:00:00+02")                  |       50
    2 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02")                  |       60
(6 rows)
```

```
postgres=# update oncall
             for portion of shift from '2026-06-10' to '2026-06-12'
             set employee = 50 where dept = 1;
UPDATE 1
```

```
postgres=# update oncall
             for portion of shift from '2026-06-10' to '2026-06-12'
             set employee = 50 where dept = 1;
UPDATE 1
postgres=# select * from oncall;
 dept |                          shift                          | employee
------+---------------------------------------------------------+----------
    1 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02") |       10
    1 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02") |       30
    2 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02") |       40
    2 | ["2026-06-08 00:00:00+02","2026-06-14 00:00:00+02") |       50
    2 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02") |       60
    1 | ["2026-06-10 00:00:00+02","2026-06-12 00:00:00+02") |       50
    1 | ["2026-06-08 00:00:00+02","2026-06-10 00:00:00+02") |       20
    1 | ["2026-06-12 00:00:00+02","2026-06-14 00:00:00+02") |       20
(8 rows)
```

```
postgres=# update oncall
             for portion of shift from '2026-06-10' to '2026-06-12'
             set employee = 50 where dept = 1;
UPDATE 1
postgres=# select * from oncall;
 dept |                          shift                          | employee
------+---------------------------------------------------------+----------
    1 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02") |       10
    1 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02") |       30
    2 | ["2026-06-01 00:00:00+02","2026-06-07 00:00:00+02") |       40
    2 | ["2026-06-08 00:00:00+02","2026-06-14 00:00:00+02") |       50
    2 | ["2026-06-15 00:00:00+02","2026-06-21 00:00:00+02") |       60
    1 | ["2026-06-10 00:00:00+02","2026-06-12 00:00:00+02") |       50
    1 | ["2026-06-08 00:00:00+02","2026-06-10 00:00:00+02") |       20
    1 | ["2026-06-12 00:00:00+02","2026-06-14 00:00:00+02") |       20
(8 rows)
```

# SQL/PGQ Property Graphs

*SQL/PGQ standard (ISO/IEC 9075-16:2023)*

*Property graphs are implemented as views over relations, no data duplication required*

*Queries are regular plans; can use indexes, be explained, mixed with regular SQL etc*

```
postgres=# create table developers (
  id serial primary key,
  name text not null,
  email text
);
CREATE TABLE

postgres=# create table follows (
  id serial primary key,
  developer integer references developers (id),
  follows integer references developers (id),
  follow_ts timestamptz default now()
);
CREATE TABLE
```

```
postgres=# insert into developers (name, email) values
('Alice', 'alice@exmaple.org'),
('Bob','bob@example.org'),
('Mallory','mallory@example.org');
INSERT 0 3

postgres=# insert into follows (developer, follows) values
(2,1),(2,3),(3,1),(1,2);
INSERT 0 4
```

```
postgres=# create property graph developer_graph
  vertex tables (
    developers label developer properties (id, name, email)
  )
  edge tables (
    follows
      source key (developer) references developers (id)
      destination key (follows) references developers (id)
      label follows
      properties (follow_ts)
);
CREATE PROPERTY GRAPH
```

```
postgres=# select * from graph_table (developer_graph
  match (a is developer where a.name = 'Alice')
  <-[f is follows]-(b is developer)
  columns (b.name as followers));

 followers
-----------
 Bob
 Mallory
(2 rows)
```

```
postgres=# select * from graph_table (developer_graph
  match (a is developer where a.name = 'Alice')
  -[f is follows]->(b is developer)
  columns (b.name as follows));

 follows
---------
 Bob
(1 row)
```

```
postgres=# select followers, count(*)
  from graph_table (developer_graph
  match (a is developer where a.name = 'Alice')
  <-[f is follows]-(b is developer)
  columns (b.name as followers)) group by followers;

 followers | count
-----------+-------
 Bob       |     1
 Mallory   |     1
(2 rows)
```

# SQL/PGQ Property Graphs

*Initial implementation - Useable scaffolding*

*Improvements underway for Postgres 20*

*Not a replacement for Neo4j just yet; does not support Cypher (the Apache AGE Postgres extension however does..)*

# pg_plan_advice Module

*Postgres has resisted planner hints for the longest time, arguing that the planner should get it right*

*The planner doesn't always get it right..*

*Designed as a plan stability feature, not a planner hint feature - not too much squinting required*

```
postgres=# explain (plan_advice, costs off)
  select followers, count(*)
  from graph_table (developer_graph
  match (a is developer where a.name = 'Alice')
  <-[f is follows]-(b is developer)
  columns (b.name as followers)) group by followers;
```

```
                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 GroupAggregate
   Group Key: developers_1.name
   ->  Sort
         Sort Key: developers_1.name
         ->  Nested Loop
               ->  Hash Join
                     Hash Cond: (follows.follows = developers.id)
                     ->  Seq Scan on follows
                     ->  Hash
                           ->  Seq Scan on developers
                                 Filter: (name = 'Alice'::text)
               ->  Index Scan using developers_pkey on developers developers_1
                     Index Cond: (id = follows.developer)
 Generated Plan Advice:
   JOIN_ORDER(follows developers developers#2)
   NESTED_LOOP_PLAIN(developers#2)
   HASH_JOIN(developers)
   SEQ_SCAN(follows developers)
   INDEX_SCAN(developers#2 public.developers_pkey)
   NO_GATHER(developers follows developers#2)
(20 rows)
```

# `pg_plan_advice` *Module*

*Advice can be fed back into the planner using a new GUC* `pg_plan_advice.advice`

*Partial advice can be fed back to enforce desired characteristics*

*The planner might disregard, and shall never generate a faulty plan due to advice*

```
postgres=# set pg_plan_advice.advice =
  'JOIN_ORDER(developers follows developers#2)';
SET

postgres=# explain
  select followers, count(*)
  from graph_table (developer_graph
  match (a is developer where a.name = 'Alice')
  <-[f is follows]-(b is developer)
  columns (b.name as followers)) group by followers;
```

```
                                                
```

# `pg_plan_advice` *Module*

*Advice can be additionally be set per query with the* `pg_stash_advice` *module*

*Relies on the Query ID which be obtained from* `EXPLAIN (VERBOSE)`

*Plans can change due to underlying data and statistics for a reason, caution required*

# Developer Experience Grab Bag

*Event triggers can now be written in PL/Python. A Dictionary is passed to the trigger with details on the invoked trigger*

`SELECT random(min, max)` *now supports the date and timestamp data types*

# Developer Experience Grab Bag

`GROUP BY ALL` *infers all non-aggregate columns in the query and groups on them*

*Window functions treat NULLs as regular values,* `IGNORE NULLS` *can now be used to skip NULLs for gap-free sequences*

# REMOVALS

# Removals

*RADIUS authentication is removed*

`MULE_INTERNAL` *encoding is removed - an Emacs specific encoding used before Unicode in order to mix encodings in a single document*



# POSTGRES 20: YOU?

# *Contributing to PostgreSQL*

*PostgreSQL is what it is thanks to individual contributors following their own agenda and scratching their own itches*

*No contribution too small*

*Everyone is welcome, everyone can contribute something. Postgres belongs to everyone*



# PostgreSQL Development

*Mailinglists instead of Github, publically archived and searchable*

*Very active Discord server for everything between mentoring and workshops to core development discussions*



# PostgreSQL Development

*Postgres is old, C was the systems language of choice in the 80's*

*"The power of assembly language and the convenience of.. assembly language"*

*- Denis Ritchie*

*No memory safety and weakly typed*

# PostgreSQL Development

*Postgres is old, C was the systems language of choice in the 80's*

*The power of assembly language and the convenience of... assembly language*

*- Dennis Ritchie*

*Fast and slow*



# PostgreSQL Development

*C, but not C. Postgres C*

*Managed memory and resources with automatic cleanup and resource management*

*Strong errorhandling, exceptions (kind of sort of) and a rich standard library and internal API*

*Safer, *not* safe*

```
buf = (char *) palloc(bytes_to_read + 1);
bytes_to_read = fread(buf, 1, bytes_to_read, file);

if (ferror(file))
    ereport(ERROR,
            (errcode_for_file_access(),
             errmsg("could not read file \"%s\": %m",
                    filename)));

FreeFile(file);
```

*Guaranteed on return* 

```
buf = (char *) palloc(bytes_to_read + 1);
bytes_to_read = fread(buf, 1, bytes_to_read, file);

if (ferror(file))
    ereport(ERROR,
            (errcode_for_file_access(),
             errmsg("could not read file \"%s\": %m",
                    filename)));

FreeFile(file);
```

*Guaranteed on return*



```
buf = (char *) palloc(bytes_to_read + 1);
bytes_to_read = fread(buf, 1, bytes_to_read, file);

if (ferror(file))
    ereport(ERROR,
            (errcode_for_file_access(),
             errmsg("could not read file \"%s\": %m",
                    filename)));

FreeFile(file);
```

*Does not return*



*Guaranteed on return*

```
buf = (char *) palloc(bytes_to_read + 1);
bytes_to_read = fread(buf, 1, bytes_to_read, file);

if (ferror(file))
    ereport(ERROR,
            (errcode_for_file_access(),
             errmsg("could not read file \"%s\": %m",
                    filename)));

FreeFile(file);
```

*Does not return*

*Automatically cleans up resources*

```
PG_TRY();
{
    walrcv_alter_slot(wrconn, sub->slotname,
            update_failover ? &opts.failover : NULL,
            update_two_phase ? &opts.twophase : NULL);
}
PG_FINALLY();
{
    walrcv_disconnect(wrconn);
}
PG_END_TRY();
```

# Contributing to PostgreSQL

*Contributing to PostgreSQL can be done in so many different ways*

*Code & Extensions*

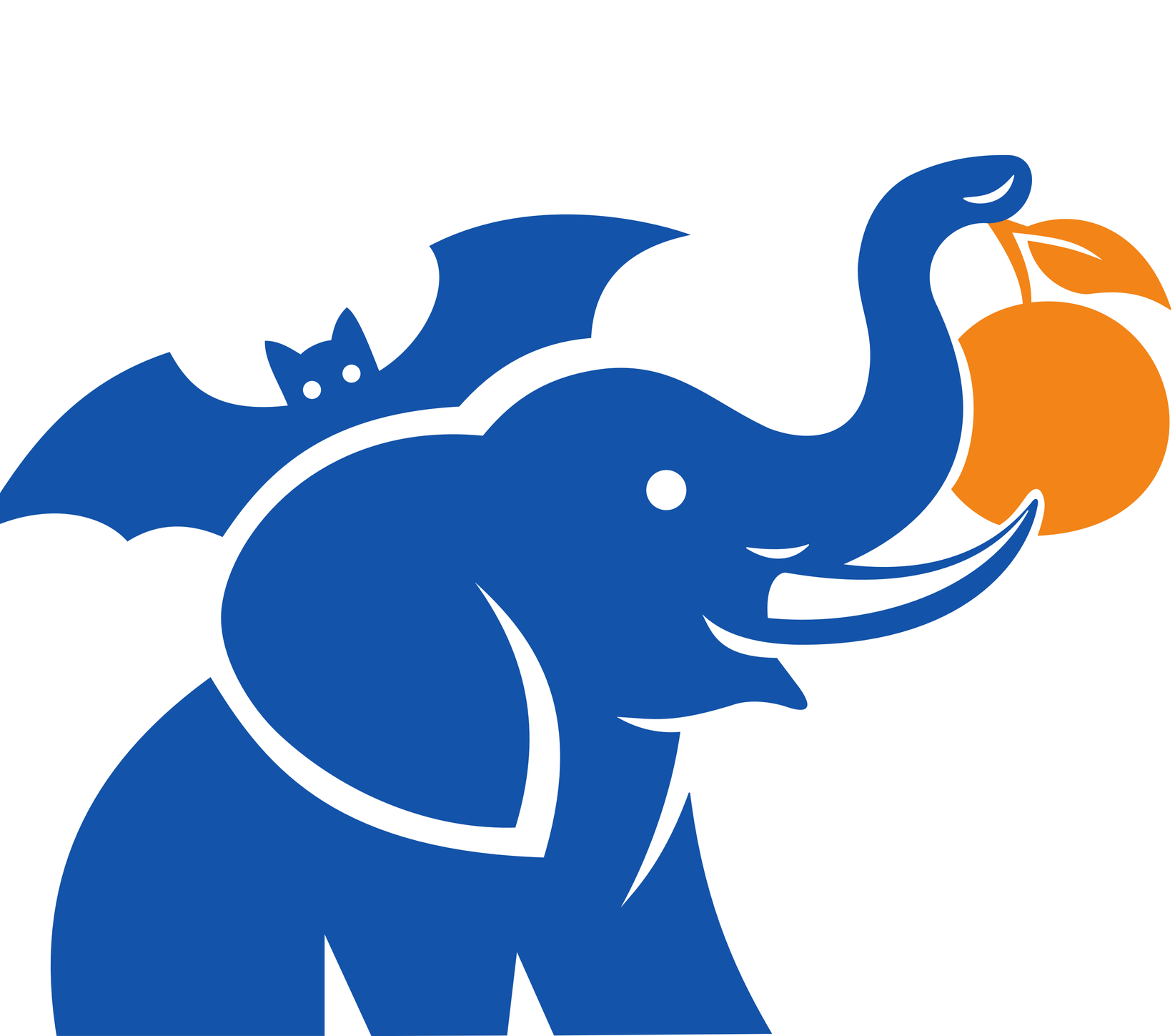
*Documentation*

*Translations*

*Web apps*

# *MORE?*


MALMÖ
POSTGRESQL
USER GROUP



# PostgreSQL Conference Europe

October 20-23

Valencia, Spain

# Have you listened to?

## TalkingPostgres.com



Microsoft

# Thank you!

daniel@yesql.se